⎈ Kubernetes RBAC

Role Bindings — Comprehensive Lecture Notes

From Basics to Advanced · Implementation · Q&A · Exercises · MCQ Quiz

Table of Contents

1. Introduction to Role Bindings
2. Why & Where We Need Role Bindings
3. Core Concepts Deep Dive
4. Implementation — All 4 Ways
5. Advanced Topics
6. Logical Questions & Answers
7. Hands-on Exercises
8. MCQ Quiz

1. Introduction to Role Bindings

> ℹ️ **What is RBAC?**
> Role-Based Access Control (RBAC) is the Kubernetes authorization mechanism that controls who can
> perform what actions on which resources. Enabled via --authorization-mode=RBAC on the API server.
> Stable (GA) since Kubernetes v1.8.

The Four RBAC Building Blocks

Object	Scope	Description
`Role`	*Namespaced*	Defines allowed API operations (verbs) on specific resources within a single namespace.
`ClusterRole`	*Cluster-wide*	Like Role but cluster-wide. Also grants access to non-namespaced resources (nodes, PVs).
`RoleBinding`	*Namespaced*	Grants a Role OR ClusterRole to subjects (users, groups, ServiceAccounts) in a namespace.
`ClusterRoleBinding`	*Cluster-wide*	Grants a ClusterRole to subjects across the entire cluster.

What is a RoleBinding?

A RoleBinding is the Kubernetes object that connects a Role (or ClusterRole) to one or more subjects. Roles define what can be done — RoleBindings define who can do it.

> ⚠️ **Key Constraint: roleRef is Immutable**
> Once a RoleBinding is created, its roleRef field cannot be changed.
> To switch the role being bound, delete the RoleBinding and create a new one.

kubectl explain rolebindings

OUTPUT

```
KIND:       RoleBinding
VERSION:    rbac.authorization.k8s.io/v1

FIELDS:
  apiVersion  <string>      rbac.authorization.k8s.io/v1
  kind        <string>      RoleBinding
  metadata    <ObjectMeta>  name, namespace, labels, annotations...
  roleRef     <RoleRef>     Role or ClusterRole being bound (IMMUTABLE)
  subjects    <[]Subject>   Users, Groups, or ServiceAccounts
```

Kubernetes Authorization Flow

REQUEST PIPELINE

```
kubectl / API Call
     |
     v
Authentication  (Who are you?)          <- Certs, OIDC, Webhooks
     |
     v
Authorization / RBAC  (Allowed?)        <- RoleBindings evaluated here
     |
     v
Admission Control  (Is it valid?)       <- Webhooks, PodSecurity...
     |
     v
etcd / Action  (Executed)
```

2. Why & Where We Need Role Bindings

✖ Without RBAC — Everything or Nothing

Without RBAC, a user either has full cluster-admin rights or no access at all.
This violates the Principle of Least Privilege (PoLP) and is a critical production security risk.

Six Reasons You Need RoleBindings

- Least Privilege Enforcement — grant developers read access to Pods without delete rights
- Multi-Team Environments — Team A owns 'frontend' namespace, Team B owns 'backend'; no cross-access
- Service Account Isolation — monitoring pod gets read-only; a controller gets write access
- Multi-Tenancy — RoleBindings enforce hard boundaries between customers sharing a cluster
- Audit & Compliance — SOC2, HIPAA, PCI-DSS require demonstrable, auditable access controls
- Blast Radius Reduction — compromised credentials limited to that identity's exact permissions

Common Use-Case Reference

Scenario	Subject Type	Binding Type	Scope
Developer accessing staging namespace	User	RoleBinding	Namespace
CI/CD pipeline deploying apps	ServiceAccount	RoleBinding	Namespace
Prometheus reading pods cluster-wide	ServiceAccount	ClusterRoleBinding	Cluster
Ops team viewing all namespaces	Group	ClusterRoleBinding	Cluster
Ingress controller managing services	ServiceAccount	ClusterRoleBinding	Cluster
App reading its own ConfigMap	ServiceAccount	RoleBinding	Namespace

3. Core Concepts Deep Dive

Subjects — The 'Who'

Kind	Description	YAML Reference
`User`	Human identity (alice, bob). Authenticated externally — not stored in Kubernetes.	`name: "alice"`
`Group`	Collection of users. system:masters = cluster-admin. Custom groups via auth provider.	`name: "dev-team"`
`ServiceAccount`	Identity for pods/processes inside the cluster. A real Kubernetes object with a namespace.	`name: "my-sa"` `namespace: "default"`

Verbs — The 'What Actions'

Group	Verbs	Guidance
Read	get, list, watch	Safe to grant to developers for monitoring
Write	create, update, patch	Grant to CI/CD pipelines or operators
Delete	delete, deletecollection	Restrict — only admins in production
Special	use, bind, escalate, *	Avoid entirely unless absolutely required

Common API Groups & Resources

```
  API GROUPS

  Core group    (apiGroup: "")              pods, services, configmaps, secrets,
                                             serviceaccounts, nodes, events,
endpoints
  apps          (apiGroup: "apps")          deployments, replicasets,
statefulsets, daemonsets
  batch         (apiGroup: "batch")         jobs, cronjobs
  RBAC          (apiGroup: "rbac.authorization.k8s.io")  roles, rolebindings,
clusterroles
  networking    (apiGroup: "networking.k8s.io")  ingresses, networkpolicies
```

RoleBinding vs ClusterRoleBinding

Feature	RoleBinding	ClusterRoleBinding
Scope	Single namespace	Entire cluster
Can bind a Role?	Yes (same namespace)	No
Can bind a ClusterRole?	Yes (scoped to namespace)	Yes (cluster-wide)
Non-namespaced resources	No	Yes (nodes, PVs, etc.)
Best use case	Namespace-level access	Cluster-wide access

> ✦ **Power Pattern: ClusterRole + RoleBinding = Reusable Permissions**

Define a ClusterRole once (e.g., 'pod-reader') and reuse it across many namespaces via individual namespace-scoped RoleBindings. No duplicating Role definitions.

Built-in ClusterRoles

ClusterRole	Access Level	Notes
cluster-admin	Full cluster control	Bypasses all checks. Never grant to workloads.
admin	Full namespace control	Cannot modify the namespace itself.
edit	Read/write most resources	Cannot manage Roles or RoleBindings.
view	Read-only access	Cannot see Secrets.

4. Implementation — All 4 Ways

Method 1: YAML Manifests

Step 1 — Create a Role

simple-role.yaml

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: default
rules:
  - apiGroups: [""]      # "" = core API group
    resources: ["pods"]
    verbs: ["get", "watch", "list"]
```

Step 2 — Create a RoleBinding

simple-rolebinding.yaml

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: default      # Must match the Role's namespace
subjects:
  - kind: User
    name: alice            # Case-sensitive!
    apiGroup: rbac.authorization.k8s.io
  - kind: ServiceAccount
    name: my-sa
    namespace: default     # SA namespace required
  - kind: Group
    name: dev-team
    apiGroup: rbac.authorization.k8s.io
roleRef:                   # IMMUTABLE after creation
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

ClusterRoleBinding Example

cluster-rolebinding.yaml

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: ops-view-global
subjects:
  - kind: Group
    name: ops-team
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: view              # Built-in ClusterRole
```

```
  apiGroup: rbac.authorization.k8s.io
```

RoleBinding + ClusterRole (Namespace-scoped)

rolebinding-clusterrole.yaml

```yaml
# Reuse a ClusterRole but scope it to one namespace
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: bob-admin-staging
  namespace: staging       # Access is limited to this namespace
subjects:
  - kind: User
    name: bob
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole        # Referencing a ClusterRole!
  name: admin
  apiGroup: rbac.authorization.k8s.io
```

Method 2: kubectl Imperative Commands

Creating Roles

bash

```bash
# Basic role
kubectl create role pod-reader \
  --verb=get,list,watch --resource=pods -n default

# Role with specific resource names (instance-level)
kubectl create role secret-reader \
  --verb=get --resource=secrets \
  --resource-name=my-secret -n production

# Role for multiple resources
kubectl create role app-manager \
  --verb=get,list,watch,create,update,patch,delete \
  --resource=pods,deployments,services -n myapp
```

Creating RoleBindings

bash

```bash
# Bind to a user
kubectl create rolebinding read-pods \
  --role=pod-reader --user=alice -n default

# Bind to a ServiceAccount
kubectl create rolebinding sa-binding \
  --role=pod-reader --serviceaccount=default:my-sa -n default

# Bind to a group
kubectl create rolebinding dev-binding \
  --role=pod-reader --group=dev-team -n default

# Bind a ClusterRole in a specific namespace
```

```
kubectl create rolebinding bob-admin \
  --clusterrole=admin --user=bob -n staging

# Create a ClusterRoleBinding
kubectl create clusterrolebinding ops-view \
  --clusterrole=view --group=ops-team
```

Method 3: Dry Run & Verify

bash

```bash
# Client-side dry-run (no API call)
kubectl create rolebinding my-binding \
  --role=pod-reader --user=alice -n default \
  --dry-run=client -o yaml

# Server-side dry-run (validates against API server)
kubectl create rolebinding my-binding \
  --role=pod-reader --user=alice -n default \
  --dry-run=server -o yaml

# Verify permissions after binding
kubectl auth can-i list pods --as=alice -n default
# Before: no    After: yes

# List everything a user can do
kubectl auth can-i --list --as=alice -n default

# Check a ServiceAccount
kubectl auth can-i list pods \
  --as=system:serviceaccount:default:my-sa -n default
```

Method 4: Advanced Patterns

Pattern 1 — Aggregated ClusterRoles

aggregated-clusterrole.yaml

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: monitoring-aggregate
aggregationRule:
  clusterRoleSelectors:
    - matchLabels:
        rbac.example.com/aggregate-to-monitoring: "true"
rules: []  # Auto-populated from matching ClusterRoles
---
kind: ClusterRole
metadata:
  name: pod-metrics-reader
  labels:
    rbac.example.com/aggregate-to-monitoring: "true"  # Pulled in automatically
rules:
  - apiGroups: ["metrics.k8s.io"]
    resources: ["pods"]
    verbs: ["get", "list"]
```

Pattern 2 — Cross-Namespace ServiceAccount Access

cross-namespace.yaml

```yaml
# Allow SA in 'monitoring' to read pods in 'production'
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: allow-monitoring-sa
  namespace: production         # Access is granted IN production
subjects:
  - kind: ServiceAccount
    name: prometheus
    namespace: monitoring        # SA lives in monitoring namespace
roleRef:
  kind: ClusterRole
  name: view
  apiGroup: rbac.authorization.k8s.io
```

Pattern 3 — RBAC for CRDs

crd-rbac.yaml

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: crd-operator
rules:
  - apiGroups: ["mycompany.io"]
    resources: ["myresources", "myresources/status"]
    verbs: ["get","list","watch","create","update","patch"]
  - apiGroups: ["apiextensions.k8s.io"]
    resources: ["customresourcedefinitions"]
    verbs: ["get", "list"]
```

5. Advanced Topics

1. Privilege Escalation Prevention

✖ Escalation Risk

A user CANNOT create a Role with more permissions than they themselves have. They also cannot create a RoleBinding granting permissions they don't possess — unless they hold the 'escalate' or 'bind' verb on the relevant role resource.

2. ResourceNames — Instance-Level Control

resourcenames-role.yaml

```
rules:
  - apiGroups: [""]
    resources: ["secrets"]
    resourceNames: ["app-db-password", "app-tls-cert"]  # Only these two
    verbs: ["get"]
# NOTE: list and watch CANNOT be restricted by resourceNames
# because they return collections, not individual instances.
```

3. System Special Groups

Group	Meaning	Risk Level
system:masters	Bypasses ALL authorization checks	CRITICAL — never use for workloads
system:authenticated	All authenticated users	Low — fine for read-only ClusterRoles
system:unauthenticated	Unauthenticated requests	HIGH — avoid granting anything
system:serviceaccounts	All SAs in the cluster	Medium — scope carefully
system:serviceaccounts:<ns>	All SAs in a specific namespace	Low — useful for namespace isolation

4. RBAC Best Practices

✔ Use namespaced Roles + RoleBindings over ClusterRoles wherever possible
✔ Create dedicated ServiceAccounts per workload; never share the 'default' SA
✔ Prefer built-in ClusterRoles (view, edit, admin) with RoleBinding, not ClusterRoleBinding
✔ Use resourceNames to restrict access to specific secret/configmap instances
✔ Audit regularly: kubectl auth can-i --list and tools like rbac-lookup, kubectl-who-can
✖ Never use wildcards (*) in production — they grant access to future resources too
✖ Never bind system:masters to non-admin service accounts
✖ Avoid granting 'escalate' or 'bind' verbs — they allow privilege escalation
✖ Never grant 'secrets list/watch' unless absolutely necessary

5. Troubleshooting RBAC Denials

bash — debug steps

```bash
# Error you'll see:
# Error from server (Forbidden): pods is forbidden:
# User "alice" cannot list resource "pods" in namespace "default"

# Step 1: Verify binding exists
kubectl get rolebindings -n default | grep alice

# Step 2: Describe the binding
kubectl describe rolebinding read-pods -n default

# Step 3: Inspect the role rules
kubectl describe role pod-reader -n default

# Step 4: Test the permission directly
kubectl auth can-i list pods --as=alice -n default

# Step 5: List all SA permissions
kubectl auth can-i --list --as=system:serviceaccount:default:my-sa
```

6. Logical Questions & Answers

Q1. What is the difference between a Role and a ClusterRole?

A: A Role is namespaced — grants permissions within a single namespace. A ClusterRole is cluster-wide — it can grant permissions on non-namespaced resources (nodes, PVs) or across all namespaces. A ClusterRole can also be bound within a namespace via a RoleBinding.

Q2. Can a RoleBinding reference a ClusterRole?

A: Yes. A RoleBinding can reference a ClusterRole, but the permissions are still scoped to the namespace of the RoleBinding. This is the key reusability pattern — define the ClusterRole once, bind it per-namespace.

Q3. Why is roleRef immutable in a RoleBinding?

A: To prevent privilege escalation. If you could change the role being bound, an attacker with write access to RoleBindings could silently escalate privileges. To change the role, delete the binding and recreate it.

Q4. What happens if a RoleBinding subject doesn't exist yet?

A: Nothing bad — the binding is created without error. Kubernetes doesn't validate subject existence at creation time. Once the subject is created and authenticates, permissions apply automatically.

Q5. How do you give a ServiceAccount in namespace A access to resources in namespace B?

A: Create a RoleBinding IN namespace B, with the subject referencing the SA in namespace A (kind: ServiceAccount, name: <sa-name>, namespace: A). The permissions are scoped to namespace B.

Q6. Can you explicitly deny actions with RBAC?

A: No. Kubernetes RBAC is purely additive. There are no deny rules. If no binding grants an action, it is denied by default. To restrict access, ensure no binding grants that permission.

Q7. What does aggregationRule do in a ClusterRole?

A: It dynamically populates a ClusterRole's rules by combining rules from other ClusterRoles whose labels match the given selectors. Used by built-in roles like 'admin' and 'edit' to allow CRD owners to extend them.

Q8. What does kubectl auth can-i check exactly?

A: It sends a SubjectAccessReview (or SelfSubjectAccessReview) to the API server to test whether a subject can perform a verb on a resource. It respects ALL authorization modes — RBAC, webhooks, etc.

7. Hands-on Exercises

[Beginner] Grant Alice read-only pod access in 'dev' namespace

1. Create a namespace called 'dev'
2. Create Role 'pod-reader' with get, list, watch on pods
3. Bind the role to user 'alice'
4. Verify: kubectl auth can-i list pods --as=alice -n dev

Solution:

```
kubectl create namespace dev
kubectl create role pod-reader \
  --verb=get,list,watch --resource=pods -n dev
kubectl create rolebinding alice-pod-reader \
  --role=pod-reader --user=alice -n dev
kubectl auth can-i list pods --as=alice -n dev  # yes
```

[Beginner] ServiceAccount with deployment management access

1. Create SA 'deployer' in namespace 'staging'
2. Create Role 'deployment-manager' with full deployments access
3. Bind the role to the SA
4. Verify the SA cannot delete pods

Solution:

```
kubectl create serviceaccount deployer -n staging
kubectl create role deployment-manager \
  --verb=get,list,watch,create,update,patch,delete \
  --resource=deployments -n staging
kubectl create rolebinding deployer-binding \
  --role=deployment-manager \
  --serviceaccount=staging:deployer -n staging
kubectl auth can-i delete pods \
  --as=system:serviceaccount:staging:deployer -n staging  # no
```

[Intermediate] Reuse a ClusterRole across multiple namespaces

1. Create ClusterRole 'ns-reader' with get/list/watch on pods and services
2. Bind it to 'bob' in namespace 'team-a' only
3. Bind it to 'carol' in namespace 'team-b' only
4. Verify bob cannot access team-b

Solution:

```
kubectl create clusterrole ns-reader \
  --verb=get,list,watch --resource=pods,services
kubectl create rolebinding bob-reader \
  --clusterrole=ns-reader --user=bob -n team-a
kubectl create rolebinding carol-reader \
  --clusterrole=ns-reader --user=carol -n team-b
kubectl auth can-i list pods --as=bob -n team-b  # no
```

[Intermediate] Cross-namespace ServiceAccount access

1. Create namespace 'monitoring' and SA 'prometheus' in it

2. Grant read access to pods in ALL namespaces
3. Use ClusterRoleBinding with built-in 'view' ClusterRole

Solution:

```
kubectl create namespace monitoring
kubectl create serviceaccount prometheus -n monitoring
kubectl create clusterrolebinding prometheus-view \
  --clusterrole=view \
  --serviceaccount=monitoring:prometheus
```

[Advanced] Least-privilege RBAC for a custom Kubernetes operator

1. Create ClusterRole managing CRD 'myapps.example.io'
2. Allow reading ConfigMaps, get-only Secrets, and creating Events
3. Create SA 'my-operator' in 'operators' namespace
4. Bind with ClusterRoleBinding for cluster-wide CRD access

Solution:

```
kubectl create namespace operators
kubectl create serviceaccount my-operator -n operators
# Apply ClusterRole via YAML:
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: my-operator-role
rules:
- apiGroups: ["example.io"]
  resources: ["myapps","myapps/status"]
  verbs: ["get","list","watch","create","update","patch","delete"]
- apiGroups: [""]
  resources: ["configmaps"]  verbs: ["get","list","watch"]
- apiGroups: [""]  resources: ["secrets"]  verbs: ["get"]
- apiGroups: [""]  resources: ["events"]   verbs: ["create","patch"]
kubectl create clusterrolebinding my-operator-binding \
  --clusterrole=my-operator-role \
  --serviceaccount=operators:my-operator
```

8. MCQ Quiz

Q1. Which Kubernetes object links a Role to a User?

A. Role
B. ClusterRole
✔ **C. RoleBinding**
D. ServiceAccount

Explanation: *A RoleBinding grants the permissions defined in a Role to subjects (users, groups, or service accounts).*

Q2. A RoleBinding can reference which of the following?

A. Only a Role in the same namespace
B. Only a ClusterRole
✔ **C. A Role in the same namespace OR a ClusterRole**
D. Any Role in any namespace

Explanation: *A RoleBinding can reference a Role in its namespace OR a ClusterRole, but permissions are always scoped to the RoleBinding's namespace.*

Q3. What happens to roleRef after a RoleBinding is created?

A. Updated with kubectl edit
✔ **B. Immutable — cannot be changed**
C. Auto-updates when the Role changes
D. Patchable with kubectl patch

Explanation: *roleRef is immutable. To change it, delete the RoleBinding and create a new one.*

Q4. Which built-in ClusterRole gives full cluster control?

A. admin
B. edit
✔ **C. cluster-admin**
D. superuser

Explanation: *cluster-admin grants full control over all resources in all namespaces and bypasses all authorization checks.*

Q5. SA 'prometheus' in 'monitoring' needs to read pods in 'production'. Best approach?

A. RoleBinding in 'monitoring'
B. RoleBinding in 'production' with monitoring SA as subject
C. ClusterRoleBinding for prometheus SA
✔ **D. Both B and C are valid**

Explanation: *B scopes access to 'production' only. C gives cluster-wide access. Both are valid depending on the requirement.*

Q6. RBAC in Kubernetes follows which model?

✔ **A. Allow-only (additive)**
B. Deny-only
C. Both allow and deny
D. Default-allow with denies

Explanation: *RBAC is purely additive. All access is denied unless explicitly granted by a binding.*

Q7. Which verbs CANNOT be restricted by resourceNames?

A. get
B. update
✔ **C. list and watch**
D. delete

Explanation: *list and watch return resource collections, not individual items, so resourceNames cannot apply to them.*

Q8. What does aggregationRule in a ClusterRole do?

A. Merges RoleBindings
✔ **B. Auto-combines rules from ClusterRoles matching label selectors**
C. Makes a ClusterRole multi-namespace
D. Aggregates audit logs

Explanation: *aggregationRule dynamically populates a ClusterRole's rules from other ClusterRoles whose labels match the selectors.*

Q9. Which group should you NEVER grant permissions to in production?

A. system:authenticated
B. system:serviceaccounts:default
✔ **C. system:masters**
D. dev-team

Explanation: *system:masters bypasses all authorization — equivalent to cluster-admin. Never use it for workloads.*

Q10. A user with 'edit' ClusterRole tries to create a RoleBinding. What happens?

A. RoleBinding created successfully
✔ **B. Denied — edit excludes RBAC management**
C. Created with limited scope
D. Requires admin approval

Explanation: *The 'edit' ClusterRole intentionally omits Roles and RoleBindings management to prevent privilege escalation.*